Student Name(s)
Donovan Chen, 28743666, donovaac@uci.edu
Carl Salazar, 67685131, cpsalaz1@uci.edu
Isai Perez, 75292336, isaip@uci.edu

Link to Project Github Link: https://github.com/DonChenn/RedditSentimentAnalysis

Trained Political Models: https://huggingface.co/carldgoat/PoliticalModels/tree/main

Trained Sentiment Model: https://huggingface.co/bagwet/SentimentModel/tree/main

Trained Topic Classifier Model: https://huggingface.co/isaip617/Topic_classifier_models

1. Project Summary (1 or 2 paragraphs)

This project investigates sentiment analysis by analyzing Reddit posts to classify both political orientation and sentiment polarity. We trained a political classifier using the "Liberals vs Conservatives" dataset and a sentiment classifier using the GoEmotions dataset, while employing unsupervised clustering (BERTopic) to identify topics. We compared baseline and advanced methods using classification accuracy, F1-score, and real-time inference latency to evaluate a live application that visualizes sentiment disparity between political groups. Once all three models were trained, we merged them into one unified pipeline to create a combined model that labels the topic, defines the sentiment, and classifies the political standing of a given post.

2. Data Sets [typically 1 page or less*]

Liberals vs Conservatives on Reddit Dataset is a collection of 13,000 Reddit posts gathered from politically leaning subreddits. Each post is labeled as Liberal or Conservative based on the subreddit it originated from. The dataset contains 6 features: post title, post text body (only contains text in about ⅓ of the data), subreddit name, upvote score, number of comments, and date created. The post text body only contains text for about ⅕ (18.89%) of the examples, with the rest only containing the text title. The text bodies that are present are an average of 183 words long. The vocabulary is informal and conversational, reflecting the nature of Reddit discourse on political topics. The class distribution is imbalanced, with 8,319 Liberal posts and 4,535 Conservative posts. We will use undersampling to balance the classes and an 80:10:10 split for training, validation, and test sets. For the topic classification model, we combined the text form post title and post text body in order to allow the model to have more text to train on.

The dataset can be found here:
https://www.kaggle.com/datasets/neelgajare/liberals-vs-conservatives-on-reddit-13000-posts

Google GoEmotions is a dataset curated from Reddit comments that contains 58,009 human annotated examples, collected and labeled by Google researchers. Reddit comments are generally short and the dataset filters comments between 10 to 30 tokens, with an average length of roughly 15 to 20 words per comment. We utilized a 80:10:10 split for our training, validation, and test sets, resulting in around 46,400 training, 5,800 validation, and 5,427 test examples. Each comment is multi-labeled with any of these: admiration, amusement, anger, annoyance, approval, caring, confusion, curiosity, desire, disappointment, disapproval, disgust, embarrassment, excitement, fear, gratitude, grief, joy, love, nervousness, neutral, optimism, pride, realization, relief, remorse, sadness, and surprise. In cases where a sample has no active emotion labels, the "neutral" label acts as a catch-all. We mapped the 28 specific labels into three broad

categories: Positive, Negative, or Neutral. The Positive bucket encompasses 12 emotions (admiration, amusement, approval, caring, desire, excitement, gratitude, joy, love, optimism, pride, and relief), the Negative bucket includes 11 emotions (anger, annoyance, disappointment, disapproval, disgust, embarrassment, fear, grief, nervousness, remorse, and sadness), and the Neutral bucket contains 5 emotions (neutral, realization, surprise, curiosity, and confusion). Because a single Reddit comment can have multiple overlapping emotion labels, we calculated the final ground truth bucket for each sample by taking a majority vote over its active emotion labels. The dataset has a significant class imbalance, with a bucketed distribution of approximately 55% Positive, 33% Neutral, and 12% Negative samples. This imbalanced distribution remains consistent across the training, validation, and test splits.

The dataset can be found here:
https://github.com/google-research/google-research/tree/master/goemotions

1 million Reddit comments from 40 subreddits is a collection of 1 million posts gathered from reddits 40 most frequented subreddits during May of 2019. Each subreddit contains 25,000 comments each one containing the subreddit the comment is from, the comment content, the controversiality, and a score based on upvotes and downvotes. This data set was used for Training the Topic classification model. Since we are focusing on political topics we only used the 25,000 comments in the r/politics subreddit. For training we only focused on using the comment content

The dataset can be found here:
https://www.kaggle.com/datasets/smagnan/1-million-reddit-comments-from-40-subreddits

Scraped Reddit Test Dataset is a collection of 5,960 comments scraped from politically active subreddits (r/Conservative, r/Liberal) using the Reddit API in February–March 2026. Each comment was collected with its source subreddit, which served as the ground-truth political leaning label (Liberal or Conservative). Because no publicly available dataset contains all three labels (sentiment, topic, and political leaning) simultaneously for the same comments, we constructed this dataset ourselves to evaluate our combined pipeline. Sentiment and topic labels were generated automatically using Gemini 2.0 Flash through a structured labeling pipeline, which prompted the model to assign one of the 28 GoEmotions labels and one of the 33 topic categories per comment. This dataset was used exclusively for final combined model evaluation and was not used in any model training.

3. Technical Approach [typically 1.5 to 2 pages]

Political Classification:

We will use TF-IDF + LogReg and Zero Shot Political Bias Bert as baselines for our political classifier. For our explorational model, we will take the pre-trained Political Bias Bert and fine tune it on our Reddit Liberals vs Conservatives dataset. After that, we will add hyperparameter tuning to this model to create our final Political Bias Bert model. To address the class balance in our Reddit Liberals vs Conservatives dataset, we implemented undersampling on the majority class to make both classes balanced.

- **Baselines: TF-IDF + LogReg & Zero Shot Political Bias Bert**
    - TF-IDF was used as the main comparative baseline and trained on our Reddit Liberals vs Conservatives dataset
    - Zero Shot Political Bias Bert was used as a rough baseline to be improved upon later with various different methods (normal BERT was also tested but performed worse than pBB in Zero Shot so we chose to build off of pBB instead)

    - For Zero Shot PBB, we needed to filter out the neutral classifications because our dataset only has Liberal and Conservative labels (it was pre-trained on 3 labels)
- **Exploratory Model: Fine Tuned + Hyperparameter Tuned Political Bias Bert**
    - Our final model that built off the rough foundation provided by pBB's pre-existing political knowledge
    - Fine tuned on our Reddit Liberals vs Conservatives dataset
    - Fine-tuning necessary for pBB because Reddit Liberals vs Conservatives dataset has only two labels which we want to classify, while the original pBB was pre-trained to classify on three labels; about 40% of the predictions fell into the neutral category, heavily shrinking the sample size of the other classes
    - When ignoring the neutral class and renormalizing probabilities, original pBB only achieved 53% on a balanced dataset, barely an improvement over guessing; the fact that it was trained on three classes seemed to have weakened pBB's classification over the two remaining classes (but still an improvement over zero shot bert which simply guessed yielding 50% accuracy, so we expanded off pBB instead)
    - Hyperparameter tuned extensively on the following parameters: learning rate, epochs , dropout, sequence length, batch size, weight decay, gradient clipping, warmup steps, warmup scheduler type

Sentiment Classification:

- **Baselines: VADER & Few-Shot (Gemini)**
    - VADER is a lightweight, rule-based lexicon model that requires no training. It computes a compound polarity score where thresholds dictate the class: scores ≥ 0.05 are classified as Positive, scores ≤ -0.05 as Negative, and the remainder as Neutral. While VADER is extremely fast, achieving an inference latency of 0.03 ms/sample, it inherently fails to capture nuanced context or sarcasm.
    - For our Few-Shot baseline, we utilized Google Generative AI (Gemini 2.0 Flash). The exact prompt structure consisted of a system instruction defining the Positive, Negative, and Neutral buckets, followed by 5 representative examples, pairing a Reddit comment to its expected bucket. For example:
        - Comment: "I absolutely love the new design! Great job to the devs."
            - "Sentiment: Positive\n\n"
        - "Comment: This is the worst update ever, completely broken and unplayable"
            - Sentiment: Negative"
    - The model was called with a temperature of 0.0 for determinism and instructed to return "Positive", "Negative", or "Neutral". While effective at leveraging context, API rate limiting of one second per call made the latency to roughly 2,380 ms/sample, making it impractical for our live production application.
- **Exploratory Model: CNN with RoBERTa**
    - Trained on the ~46,400 GoEmotions training samples, input sequences were processed using a RobertaTokenizer (with padding and truncation applied up to a max_length of 128) and passed into a frozen embedding layer loaded from the cardiffnlp/twitter-roberta-base model (768 dimensions).

    - Because these embeddings were completely frozen, the model only learned the downstream convolutional and linear layer weights and could not adapt the base embeddings to the emotion task.
    - The architecture fed these embeddings into three parallel Conv1D filters (kernel sizes of 3, 4, and 5 utilizing 100 filters each) to extract n-gram features. This was followed by a ReLU activation function, max pooling over the sequence, and concatenation into a 300-dimensional vector.
    - The features were passed through a dropout layer (p=0.5) and a fully connected linear layer that mapped the representation to the 28 emotion classes. The network was optimized over 10 epochs using the Adam optimizer (learning rate = 0.001, batch size = 32) against a BCEWithLogitsLoss function that utilized linear per-class positive weights (neg_count/pos_count).
- **Final Model: Fine-Tuned RoBERTa (cardiffnlp/twitter-roberta-base)**
    - Our final model uses a full fine-tuning of the cardiffnlp/twitter-roberta-base transformer, trained on the ~46,400 GoEmotions training samples.
    - This specific base model, pretrained on ~58 million tweets, was selected because Reddit text is similarly informal, slang-heavy, and sarcastic; this domain-matched pretraining provided a significantly better initialization than standard news or academic models.
    - We updated all transformer weights and passed the [CLS] token representation into a new classification head mapping from 768 to 28 dimensions. To address severe class imbalance without over-penalizing rare classes like "grief" (which only contains 6 test samples), we optimized against BCEWithLogitsLoss using square root-dampened class weights (pos_weight = sqrt(neg_count / pos_count)). The model was trained using AdamW (learning rate = 3e-5, warmup ratio = 0.1, batch size = 16, weight decay = 0.01) over 6 epochs, and the best checkpoint was selected based on the macro F1 score on the validation set.
    - Post-training, we optimized a separate sigmoid threshold for each of the 28 emotion classes on the validation set, selecting the value that maximized per-class F1. Thresholds ranged from 0.30 for rare emotions such as surprise and relief to 0.63 for gratitude, compared to a naive uniform 0.50 cutoff.
    - For evaluation, we report both 28-class multi-label metrics and 3-bucket (Positive/Negative/Neutral) metrics. The bucketed metric is the primary comparison metric because rare classes such as grief (6 test samples) and relief (11 test samples) make 28-class macro F1 unreliable.

Topic Classification:

For our classification model, we did a comparison of a baseline model using TF-IDF and an experimental model using BERTopic.

Preprocessing:

- For the training data set that will be used on both models we combined 2 data sets. Those data sets being Liberals vs Conservatives on Reddit (13,000 posts) from kaggle and 1 million Reddit comments from 40 subreddits (just from the r/politics dataset ~ 25,000 reddit comments).

- To increase the quality of topics produced by both models we also created a data cleaning pipeline to eliminate common stop words and common political words. This is to ensure vague words that are used commonly do not take up a majority of topics.
    - Examples of words we eliminated would be **“america”**, **“politics”**, **“talk”**, **“joke”, “trump”, “joe”, “september”, “reddit”**

<u>Baseline model: TF-IDF and Non-negative Matrix Factorization</u>

- For our Baseline TF-IDF model we assign our TF-IDF vectorizer to ignore any words in our list of stopwords, to only count words that are in at least 20 comments or more to ignore typos, and to only keep track of alphabetic characters that are at least 3 chars long to ignore emojis, numbers, and short words. Then we run the model on all the comments where it assigns a score for every word for every document based on its importance on that document.
- Once our TF-IDF matrix is created we create our **Non Negative Matrix Factorization** model. We limited the number of topics created to just 50 to make naming each topic less time consuming, set it to a specific random state to get consistent results, and our initialization method to Nonnegative Double Singular Value Decomposition Average (NNDSVDA) to reduce sparsity, and a maximum iteration count of 500.
- Then we feed our TF-IDF matrix into our NMF model so that it splits the matrix into two smaller matrices, one that shows how strong each comment is to each topic cluster and the other that shows how strong each word is related to each topic cluster
- We then use the Topic-word matrix to show the first 20 words of each topic to be able to give them a readable label that is able to show the common theme between all the words.

<u>Experimental Model: Seeded BERTopic with K-means clustering</u>

- For our BERTopic model we first initialized the vectorizer to tell the BERTopic model how to handle the words that are going to be processed. We added our custom list of stopwords to be ignored by the BERTopic model, we made it so the model looks at single words as well as two word phrases, to keep words that appear in in at least 10 comments, and to only keep words that are 3 letters long and are only in the alphabet.
- Then we initialized our seeded topics list which helps the BERTopic model to better group together political topics based on the seeded topic. An example of some of the seeded topics we made would be for the topic of Gun Control. We included words such as **“gun”, “arms”, “concealed”, "shooting**". For the topic of Racial injustice we chose words like **"racism", "systemic", "blm", "black", "white", "diversity".** This lets the model know that these words are similar to one another to help improve the quality of the topics.
- We then use **Uniform Manifold Approximation and Projection (UMAP)** to reduce the dimensions of each individual comment down to just 5 dimensions. The reason for this is to help reduce noise and focus on the patterns that matter the most when defining topics. It also helps create clusters as if we have too many dimensions comments may be too far apart to create a cluster meanwhile if we have 5 it will be easier to cluster and separate comments. We also specify to look at the 15 nearest neighbors when creating its clusters. We also use cosine distance to find the nearest 15 nearest neighbors as cosine distance is not affected by length as much as Euclidean distance, so if two comments cover the same

topic but one is longer than the other, using cosine distance would still result in them being close neighbors in the graph.

- Then we set our clustering algorithm to use K-means clustering. This is because when using BERTopic without K-means clustering, BERTopic uses HDBSCAN which sets a topic label just for noise/outliers, and does not force every comment into a cluster. This would lead a majority of comments to be considered outliers. Since we want at least 49 topics we set the number of clusters to 49 and set a fixed random state. The K-means clustering algorithm will force every comment into one of the 49 topics created.
- After we run our BERTopic model with our UMAP, K-means clustering, and seeded topic list and view the results of the top words for each topic created. We then see that there are topics that are duplicates of each other

```
Index 43: ban, civil, wishing, insults, shill, hate speech, advocating,
violation, harm, permanent
Index 44: ban, civil, wishing, insults, shill, hate speech, advocating,
violation, harm, permanent
Index 45: removed, questions, explicitly, participating, reasons,
following, message, current, thank, question
Index 46: ban, civil, wishing, insults, shill, hate speech, advocating,
violation, harm, permanent
Index 47: ban, civil, wishing, insults, shill, hate speech, advocating,
violation, harm, permanent
Index 48: forums, , , , , , , , ,
```

- We then go through and merge the topics that are duplicates with each other, overlap on similar topics (such as being conversational filler), or contain little information (index 48).
- After getting our final list of topics we then assign a topic mapping to each index based on the top 20 words for each index to improve readability and evaluation for the LLM evaluation.

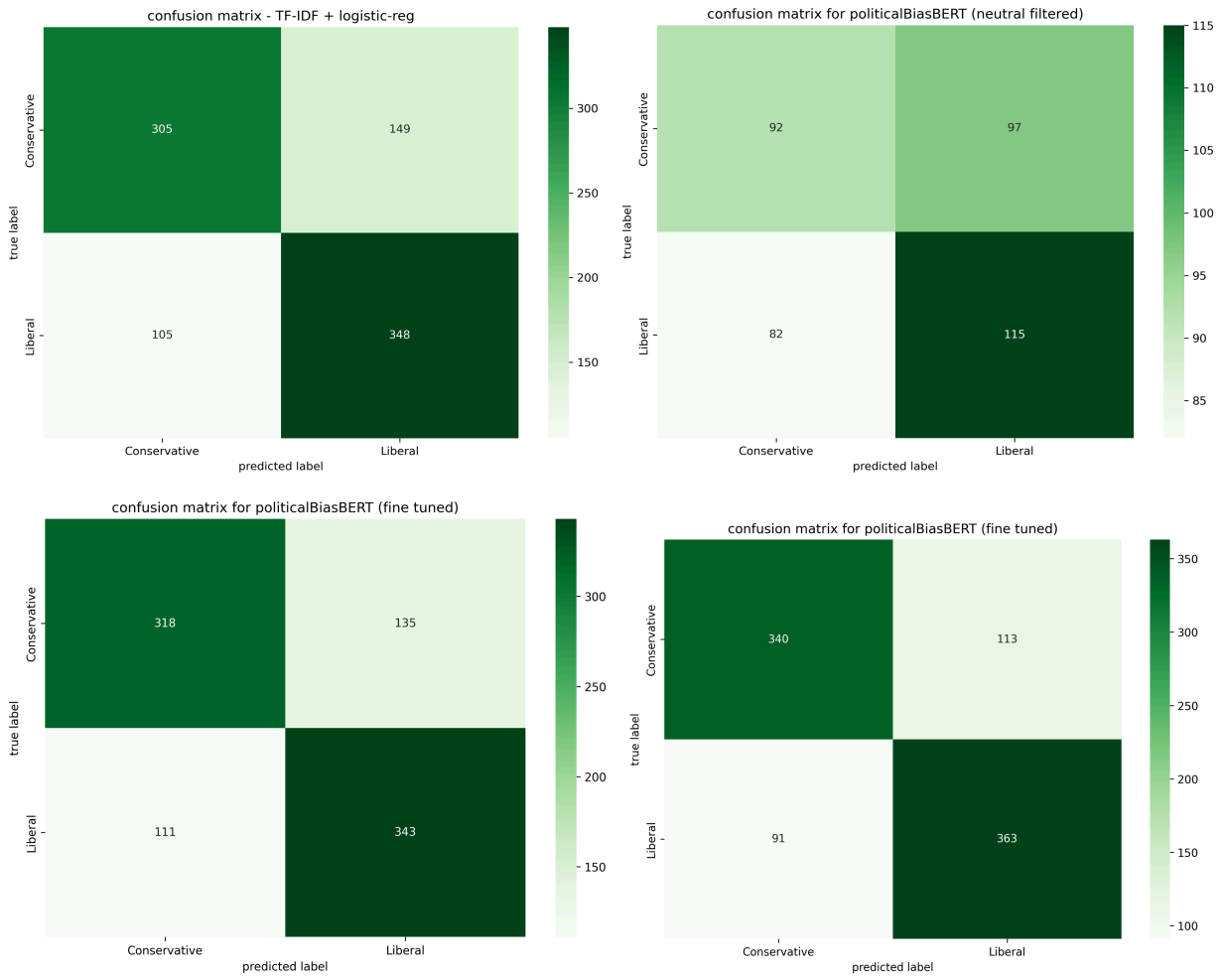
Combined Model: We will combine the best three performing models (one from each classifier) to create our final model that takes a post and identifies its label, political standing, and sentiment. It will be compared and evaluated using F1 accuracy individually and a partial credit scoring system as a whole against a combined model of our best baselines.

4. Experiments and Evaluation [at least 2 pages]

Political Model: computed latency, precision, recall, and weighted F1-score (50/50 weight since classes are balanced) for binary political labels ("Liberal" and "Conservative") on all models. Additional details below.

Model	Notes	Latency (ms/post)	Weighted F1 Score
TF-IDF + Logistic Regression (Baseline)	Fast & most competitive baseline using conventional techniques. Good at guessing simple examples without text sections. Slightly better at guessing conservative (71%) over liberal (73%) examples.	0.007	0.72

Zero-Shot politicalBiasBERT (Baseline)	High latency due to being run on CPU; model to be expanded upon. Performed worse than expected due to mismatch with dataset (extra neutral class). Barely better than guessing because the dataset is balanced (50/50). Much better at guessing conservative (61%) over liberal (40%) examples. Likely pre-trained on majority conservative data or most liberal leaning data was classified into the neutral class.	1339.3	0.53
Fine-Tuned politicalBiasBERT	Ran on GPU; good middle ground model. Immediate large improvement over zero-shot because it was fine-tuned specifically on a two class dataset (which we also evaluated on) rather than the three it was originally pre-trained to evaluate. Immediate improvement on all types of examples, but especially simpler ones that didn't include a text section. Still making notable errors on more complex examples (ex: long text section examples).	1.36	0.73
Fine-Tuned + Hyperparameter Tuned politicalBiasBERT	Ran on GPU; Most accurate but takes a bit more due to the increase of complexity and number of epochs used. Improved mostly on examples with complex, longer text sections (likely as a result of increasing max text length for model during hp tuning). Almost balanced at guessing conservative (77%) vs liberal (78%) examples.	7.48	0.78



Figures 1-4: Positive, Negative confusion matrix for TF-IDF + LOG-REG (top left), Zero Shot PBB (top right), Fine-Tuned PBB (bottom left), Fine-Tuned + Hyperparameter-Tuned PBB (bottom right)

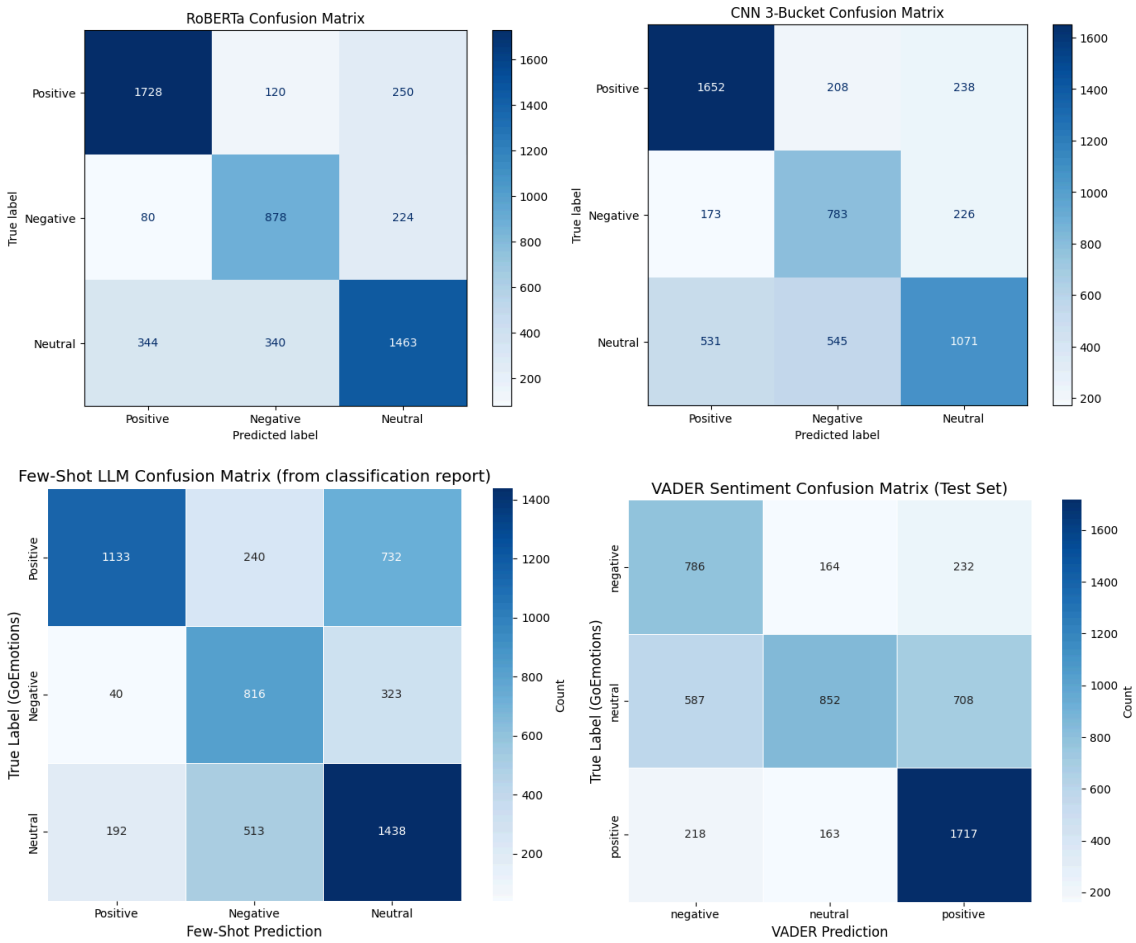
Sentiment Model: we computed latency alongside both Macro average F1 and Weighted average F1. Macro avg F1 represents the unweighted average of F1 scores across the three sentiment buckets (Positive, Negative, Neutral), treating each class equally regardless of sample count to expose minority class performance. Weighted avg F1 computes the F1 score averaged by each class's support (sample count), which better reflects overall performance proportional to the imbalanced distribution of the data. All models were evaluated on the GoEmotions test split (n=5,427 samples).

Model	Notes	Latency (ms/post)	Weighted F1 Score
VADER (Baseline)	Rule-based lexicon, no training	0.03	0.62

Few-Shot Gemini 2.0 Flash (Baseline)	Bucket definitions not provided in prompt, Gemini used its own sentiment groupings.	2380	0.62
CNN	Neural; frozen embeddings; GoEmotions train	0.85	0.65
Fine-Tuned RoBERTa (Final Model)	Full fine-tune; dampened weights; per-class thresholds	5.85	0.75

Per-Class Breakdown for Final Model (Fine-tuned RoBERTa)

Class	Precision	Recall	F1
Positive	0.80	0.82	0.81
Negative	0.66	0.74	0.70
Neutral	0.76	0.68	0.72



Figures 5-8: Positive, Negative, Neutral confusion matrix for RoBERTa (top left), CNN (top right), Few-Shot(bottom right), VADER (bottom left)

Figures illustrating the 3-bucket confusion matrices for all four models utilize a consistent vmin/vmax scale to ensure fair visual comparison across the varying distributions. Across all four models, the Negative class is consistently the hardest to classify and are mainly labeled Neutral. The language of mild negative emotions like annoyance and disappointment is often indistinguishable from neutral observation without full contextual understanding. Short or single-word comments represent a subset where all models fail.

- VADER heavily over classifies text into the Positive bucket, having a strong positive recall of 0.82 but a poor neutral recall of 0.40. This is a limitation of VADER's compound score as ambiguous or neutral text often receives a weakly positive compound score and gets pushed into the Positive bucket rather than Neutral.
- The CNN model noticeably improves neutral detection but continues to struggle with the Negative class (F1=0.58), as frozen embeddings cannot adapt to the vocabulary differences between negative emotion types like annoyance, disappointment, and disapproval.
- The Few-Shot LLM baseline demonstrated a much better neutral recall (0.67) but unexpectedly weak positive recall (0.54), having a lower F1 than the CNN despite being a complex model. The primary cause is that the Gemini prompt never defined which of the 28 GoEmotions emotions map to which bucket. Gemini classified its own prior notion of sentiment, which does not align with GoEmotions' specific mapping (GoEmotions treats surprise, realization, and curiosity as Neutral, which Gemini would not). Additionally, the 5 in context examples used generic gaming and tech comments rather than political Reddit text, leading to poor domain for the actual test distribution.
- The Fine-Tuned RoBERTa is the best model across all categories. It performs strongly on Positive text (F1=0.81) and achieves meaningful gains on the Negative class (F1=0.70), where all other models struggle due to the highly overlapping vocabulary among negative emotion sub-types. Neutral detection (F1=0.72) also improves over all baselines.

Error Analysis on Fine-Tuned RoBERTa:

- Fine-tuning RoBERTa produced the best results on complex, multi-labeled comments and politically sarcastic text where emotional meaning depends on full-sentence context. The least improvement was observed on very short comments (1–3 words), which remained difficult regardless of model complexity and may reflect label noise in the dataset.

Topic model: Since our Topic classification model is an unsupervised model we evaluated the topics based on the quality of the topics created based on the top words. Throughout each iteration of the BERTopic model our goal was to make it so the top words all shared an overall political theme to where we would be able to give it a mapping that accurately describes the overall topic pattern the model has created.

```
Index 0: Other / Irrelevant / Noise
Index 1: mueller, barr, letter, impeachment
Index 2: hes, youre, happen, thanks
Index 3: capitalism, socialism, social, socialist
Index 4: ukraine, russia, putin, russian
Index 5: barr, hes, summary, testimony
Index 6: insurance, healthcare, health, care
Index 7: sex, sexual, female, gender
Index 8: senate, mcconnell, mitch, theyre
Index 9: tax, money, taxes, rich
Index 10: shes, warren, candidate, supporters
Index 11: removal, questions, automatically, contact
Index 12: graham, lindsey, lindsey graham, hes
Index 13: black, race, white, school
Index 14: supreme, supreme court, court, candidate
Index 15: summary, report, pages, conclusions
Index 16: reagan, hes, presidency, worst
Index 17: prison, rights, law, laws
```

```
Index 0: Other / Irrelevant / Noise
Index 1: mueller, barr, impeachment, investigation
Index 2: ukraine, russia, putin, russian
Index 3: voters, elections, senate, democratic
Index 4: graham, hes, ted, hearing
Index 5: capitalism, socialism, social, socialist
Index 6: tax, money, taxes, wealth
Index 7: hes, pete, candidate, supporters
Index 8: happen, youre, joke, thanks
Index 9: feminist, gender, sex, womens
Index 10: shes, warren, emails, pelosi
Index 11: black, white, race, racism
Index 12: workers, labor, unions, strike
Index 13: prison, prisoners, jail, rights
Index 14: gun, police, guns, shooting
Index 15: insurance, healthcare, health, medicare
Index 16: summary, report, letter, conclusions
```

- The topics on the **TOP LEFT** are the result of an Unseeded BERTopic model, and the topics on the **TOP RIGHT** are the results of a Seeded BERTopic model that used 10 words per seed.
- As you can see above with the Seeded BERTopic, it added more cohesive topics such at index 18 where creates a topic around the COVID-19 pandemic with "**covid, vaccine, pandemic, mask**", public infrastructure on index 39 with "**infrastructure, transportation, roads, bridge**", and drug use with index 32 with "**marijuana, drugs, drug, weed**".

```
Index 0: talking, theyre, exactly, real, funny, mean, pretty, god, okay, way
Index 1: senate, house, mcconnell, reagan, theyre, power, white, law, democratic, run
Index 2: capitalism, socialism, social, socialist, china, communist, market, communism, democratic,
democracy
Index 3: lie, congress, lying, resign, lies, returns, fraud, oath, jail, hearing
Index 4: ted, lieu, hearing, water, buttery, movie, way, males, dick, dude
Index 5: ukraine, russia, putin, russian, war, nato, invasion, sanctions, military, conflict
Index 6: mueller, barr, letter, barrs, report, summary, investigation, special, muellers, public
Index 7: mueller, letter, public, doj, report, investigation, department, muellers, released, mueller
report
Index 8: candidate, primary, warren, poll, candidates, voters, nominee, progressive, polls, pete
Index 9: doj, perjury, congress, testify, contempt, indicted, justice, indict, sitting, mueller
Index 10: report, summary, context, title, headline, word, lie, pages, applied, redacted
Index 11: democracy, theyre, god, education, history, libs, far, shapiro, way, fear
Index 12: voters, electoral, elections, primary, ballot, polls, poll, democratic, democracy, rights
Index 13: barr, barrs, credibility, testimony, congress, lied, william barr, lying, william, impeached
Index 14: obstruction, collusion, investigation, report, evidence, justice, crime, campaign,
conspiracy, russian
Index 15: nazis, israel, fascism, nazi, leftist, fascist, social, war, jewish, freedom
Index 16: pelosi, warren, impeachment, buttigieg, candidate, nancy, campaign, pete, progressive, win
Index 17: workers, labor, unions, strike, wage, union, minimum, wages, pay, work
Index 18: graham, lindsey, lindsey graham, emails, lindsay, talking, hillarys, buttery, males, clintons
Index 19: black, white, race, racism, racist, rights, slavery, police, minorities, history
Index 20: economy, inflation, market, economic, money, growth, price, income, labor, prices
Index 21: feminist, gender, sex, female, rape, trans, male, lgbtq, sexual, feminists
Index 22: news, fox, fox news, media, fake, fake news, propaganda, cnn, conspiracy, msnbc
Index 23: climate, border, canadian, canada, immigration, climate change, oil, energy, wall, convoy
Index 24: impeachment, impeach, senate, impeached, nixon, house, convict, proceedings, office, trial
Index 25: account, content, approved, automatically, internet, organizations, contain, form, action,
site
Index 26: tax, taxes, wealth, rich, income, money, pay, billionaires, wealthy, gains
Index 27: money, corruption, funding, donors, campaign, donation, corporations, donations, corporate,
subsidies
Index 28: insurance, healthcare, health, medicare, pay, care, health insurance, health care, medical,
doctor
Index 29: abortion, baby, birth, womens, rights, child, texas, babies, children, doctors
Index 30: venezuela, coup, war, cuba, military, oil, opposition, korea, sanctions, cia
Index 31: covid, vaccine, pandemic, medical, health, mask, masks, public health, mandate, mandates
Index 32: prison, prisoners, jail, rights, prisons, law, crime, sentence, criminal, justice
Index 33: gun, guns, amendment, violence, shooting, armed, rights, mass, control, law
Index 34: police, officer, cops, shooting, officers, enforcement, safety, law, arrest, law enforcement
Index 35: court, supreme, supreme court, judge, judges, courts, prosecutors, justice, judicial, case
Index 36: land, housing, property, rent, tax, homeless, estate, market, capital, ownership
Index 37: cpac, florida, alabama, school, teachers, recall, schools, teacher, governor, california
Index 38: ban, civil, wishing, insults, shill, hate speech, advocating, violation, harm, permanent
Index 39: student, debt, college, education, loans, tuition, loan, pay, cancel, school
Index 40: marijuana, drugs, drug, weed, mental, cancer, pot, health, mental health, jail
Index 41: thank, length, updates, temporarily, news, concerning, removing, discussed, scale,
participating
Index 42: submitted, mistakes, participating, following, message, removed, thank, reason, question,
questions
Index 43: ban, civil, wishing, insults, shill, hate speech, advocating, violation, harm, permanent
Index 44: ban, civil, wishing, insults, shill, hate speech, advocating, violation, harm, permanent
Index 45: removed, questions, explicitly, participating, reasons, following, message, current, thank,
question
Index 46: ban, civil, wishing, insults, shill, hate speech, advocating, violation, harm, permanent
Index 47: ban, civil, wishing, insults, shill, hate speech, advocating, violation, harm, permanent
Index 48: forums, , , , , , , , ,
```

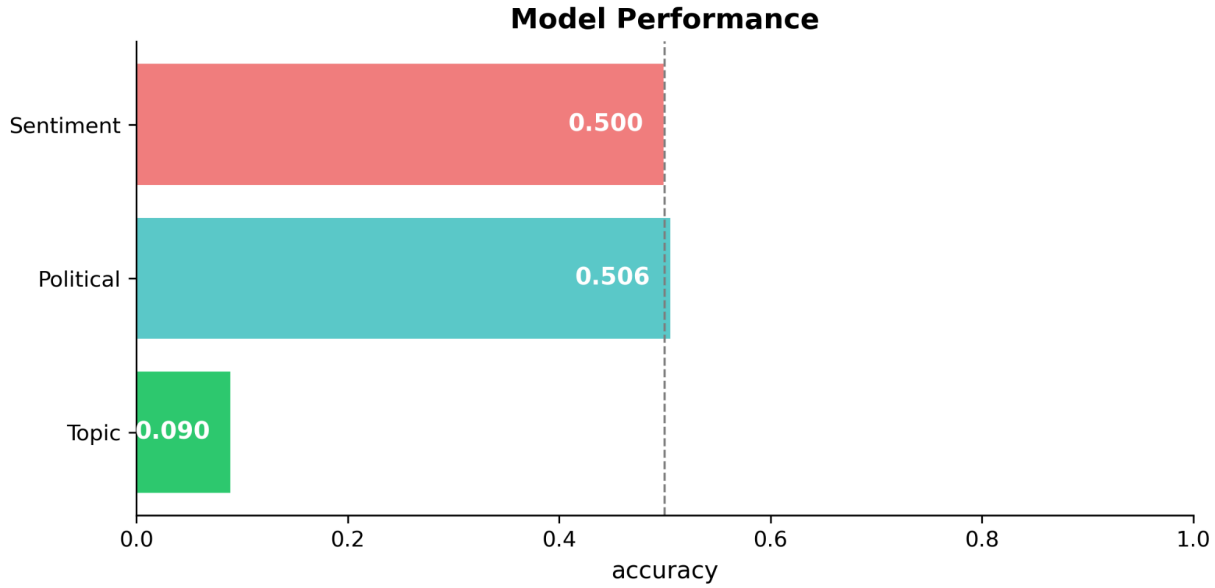
```
Index 0: mueller, barr, report, letter, investigation, doj, congress, public, summary,
special
Index 1: voters, candidate, primary, warren, candidates, poll, win, polls, democratic,
campaign
Index 2: removed, participating, thank, questions, message, following, question, mistakes,
reasons, reason
Index 3: senate, house, mcconnell, reagan, theyre, power, white, law, democratic, way
Index 4: capitalism, socialism, social, socialist, china, market, communist, democratic,
communism, democracy
Index 5: congress, lie, lying, resign, lies, returns, fraud, oath, jail, hearing
Index 6: ukraine, russia, putin, russian, war, nato, invasion, sanctions, military,
conflict
Index 7: democracy, theyre, god, education, history, far, way, shapiro, different, war
Index 8: nazis, israel, fascism, nazi, leftist, fascist, social, war, jewish, freedom
Index 9: workers, labor, unions, strike, wage, union, minimum, wages, pay, work
Index 10: black, white, race, racism, racist, rights, slavery, police, history, minorities
Index 11: economy, inflation, market, money, economic, growth, price, income, labor, value
Index 12: feminist, gender, sex, female, rape, male, trans, sexual, lgbtq, radical
Index 13: news, fox, fox news, media, fake, fake news, propaganda, cnn, conspiracy, msnbc
Index 14: climate, border, canada, canadian, immigration, climate change, oil, energy,
wall, protests
Index 15: impeachment, senate, nixon, house, office, trial, congress, criminal, crimes,
justice
Index 16: account, content, approved, automatically, internet, organizations, contain,
form, action, social
Index 17: tax, taxes, wealth, rich, income, money, pay, billionaires, wealthy, gains
Index 18: money, corruption, funding, donors, campaign, corporations, donations, corporate,
subsidies, farmers
Index 19: insurance, healthcare, health, medicare, pay, care, health insurance, health
care, medical, doctor
Index 20: abortion, baby, birth, rights, womens, child, texas, children, babies, law
Index 21: venezuela, coup, war, cuba, military, oil, opposition, sanctions, korea, cia
Index 22: ban, civil, shill, advocating, violation, harm, permanent, users, contact,
automatically
Index 23: covid, vaccine, pandemic, medical, health, mask, masks, public health, mandate,
mandates
Index 24: prison, prisoners, jail, rights, law, prisons, crime, sentence, criminal, justice
Index 25: gun, guns, amendment, violence, shooting, rights, armed, mass, law, control
Index 26: police, officer, cops, shooting, officers, safety, enforcement, law, arrest, law
enforcement
Index 27: court, supreme, supreme court, judge, judges, courts, justice, judicial, case,
resign
Index 28: land, housing, property, rent, tax, homeless, estate, market, capital, ownership
Index 29: cpac, florida, alabama, school, teachers, recall, schools, teacher, governor,
california
Index 30: student, debt, college, education, loans, tuition, loan, pay, cancel, school
Index 31: marijuana, drugs, drug, mental, cancer, pot, health, jail, mental health, legal
Index 32: thank, news, updates, temporarily, concerning, removing, discussed,
participating, scale, hello
```

- The topics created on the **LEFT** are the result of the BERTopic model after using K-means clustering and Seeding with 20 words per seed topic with No Merges. The topics created on the

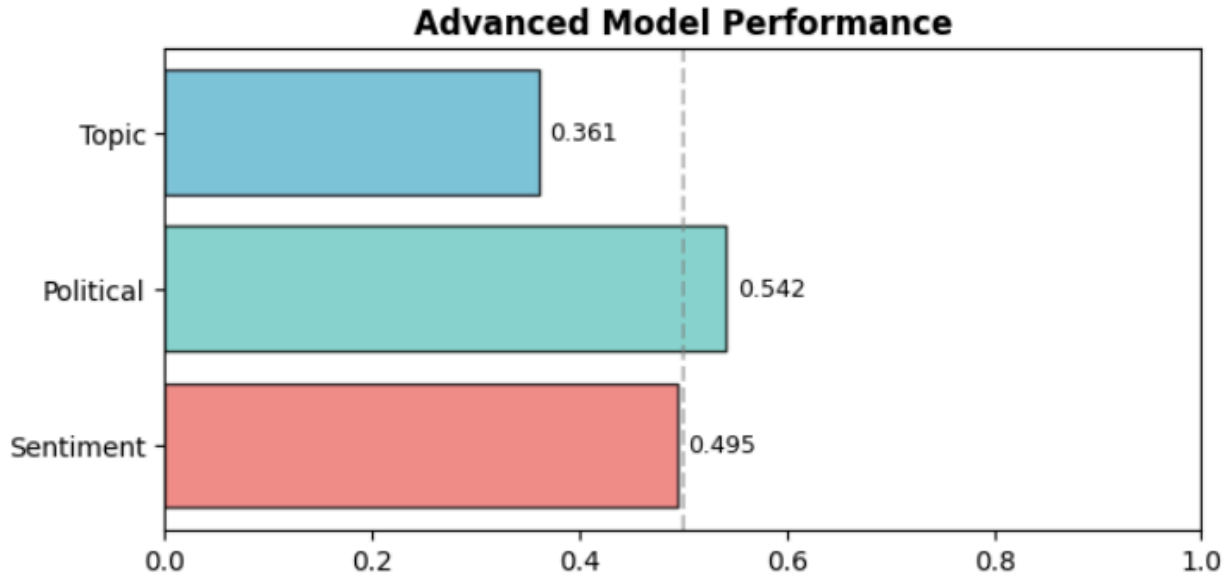
RIGHT are the result of the BERTopic model after using K-means clustering and Seeding with 20 words per seed topic after merging.

- We merged duplicate topics such as index 38, 43, 44, 46 and 47 since they all shared the same exact top words of "**ban, civil, wishing, insults, shill, ….**"
- We also merged topics 6, 13 since both of them dealt with the Mueller report and we merged topics 8 and 16 that shared the overall theme of primaries and polling.
- We merged topics 25, 41, 42, and 45 that dealt with comments that are focused on the solely on the subreddit and not on politics themselves like "**account, approved, thanks, removing, removed question**"
- After the merges the topic quality for the topics remaining are more cohesive to where we were able to give a topic label to each topic based on the 20 top words.

Combined Model: computed precision, recall, and F1-score for political, sentiment, and topic labels on a combined baseline vs combined advanced model pipeline. Models evaluated separately and together with a partial credit scoring system. Additional details below.



Above is the performance for each individual baseline model on the final dataset. The baselines used were Fine-Tuned RoBERTa for the sentiment model, TF-IDF + Log-Reg for the political model, and TF-IDF for the topic model.



Above is the performance for each individual exploratory model on the final dataset. The exploratory models used were VADER for the sentiment model, Fine-Tuned + Hyperparameter Tuned Political Bias Bert for the political model, and Seeded BERTopic with K-means clustering for the topic model.

Overall, we saw an improvement in most of our models individually, with the most notable improvement of 27% coming from the topic model. The political model improved about 4% and the sentiment model stayed about the same.

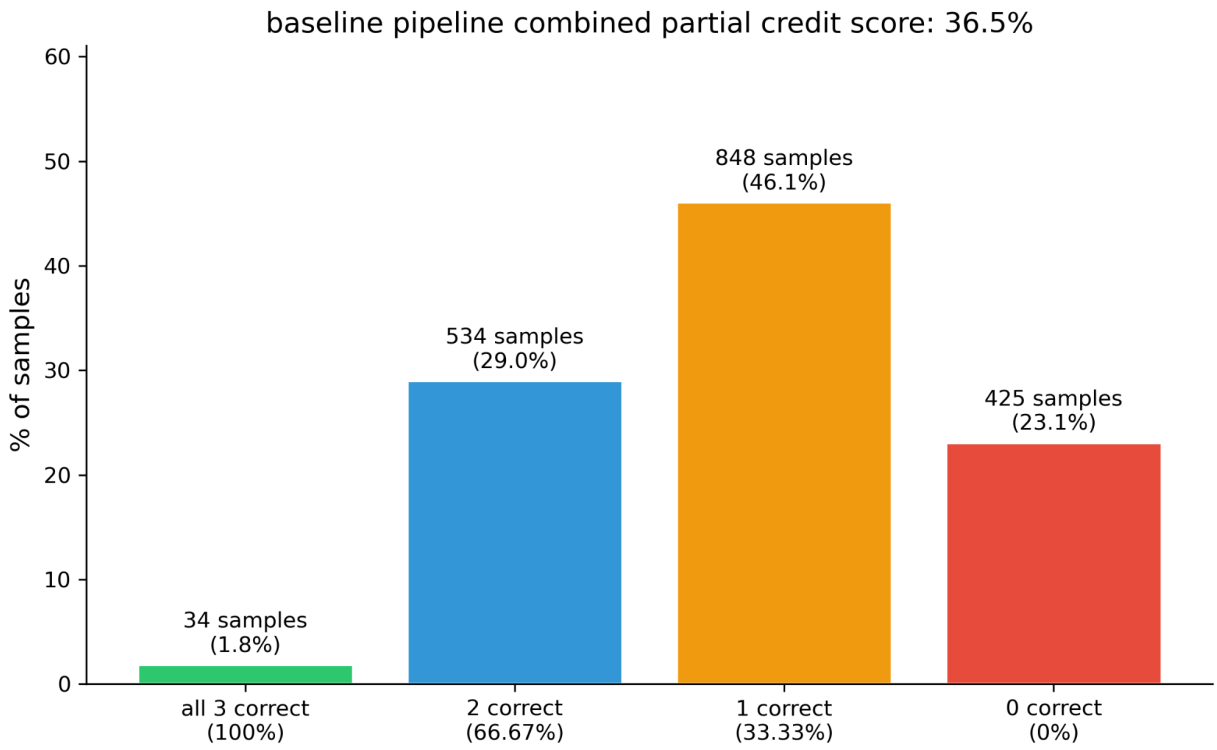
The political model showed a similar improvement in the original test set (6%), showing that the model's improvement scaled to unseen data as well. It didn't end up overfitting the training data, as that would have led to a lack of improvement on our unrelated final test set. This shows that overall, the exploratory political model was superior in terms of accuracy at the cost of being slightly slower.

However, both models performed lower on the final dataset (50% baseline, 54% exploratory) compared to their original test sets (72% and 78% respectively). This drop is likely due to the fact that the final dataset was scraped from a broader range of Liberal vs. Conservative subreddits not seen during training, meaning the models encountered new political language patterns and post styles they weren't specifically trained on. Also, the final dataset had a slight class imbalance which could have thrown off our accuracy and improvement a bit due to the fact that our model was trained on a balanced dataset. Finally, the final dataset was scraped very recently while the original Liberals vs Conservatives dataset is approximately 4 years old, meaning the model was trained on political language and topics from 2022 that may no longer reflect how people discuss politics today; this is a significant limitation given how rapidly political vocabulary evolves which likely contributed to the poorer results as a whole.

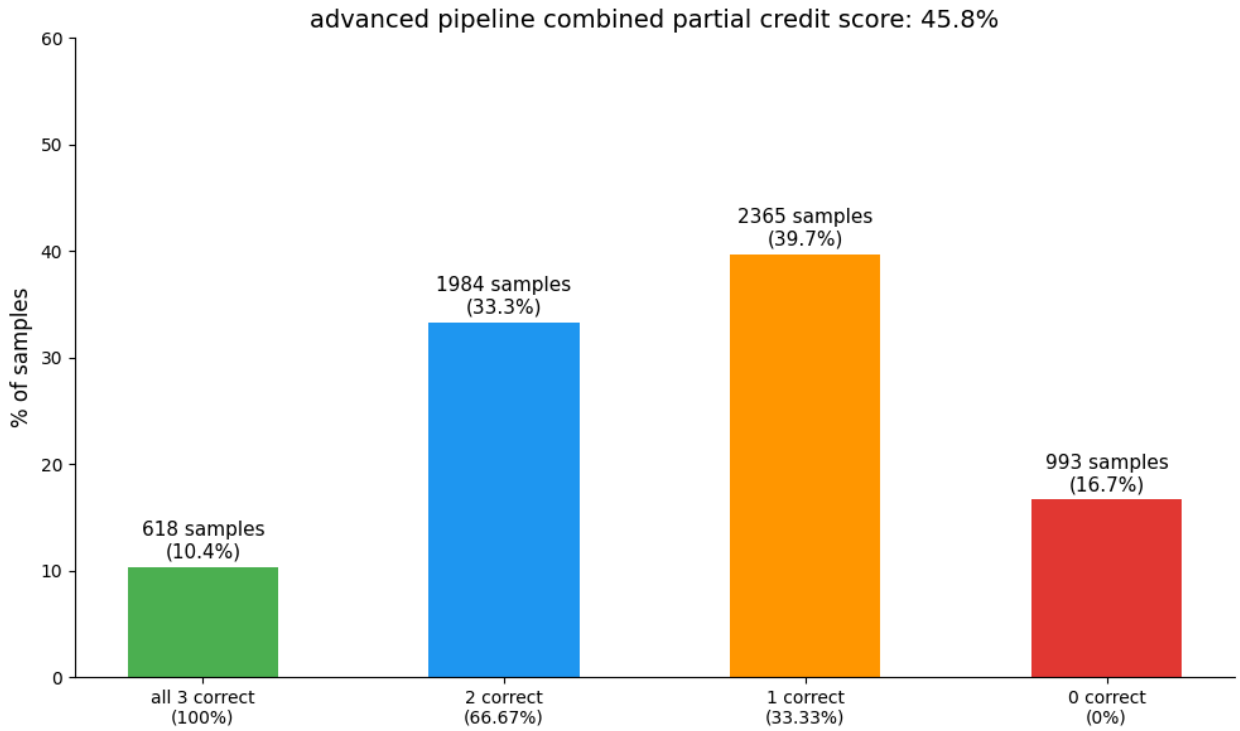
The sentiment model essentially showed no improvement on the final dataset, with the advanced Fine-Tuned RoBERTa (49.5% accuracy) performing nearly identically to the VADER baseline (50.0%). This stands in contrast to our findings on the isolated GoEmotions test set, where RoBERTa substantially outperformed VADER (75% vs. 62%). The primary cause for this collapse in relative performance is a fundamental labeling mismatch. The final dataset's ground truth labels were generated by Gemini, which classified sentiment using its own prior definitions rather than the GoEmotions specific bucket mapping. RoBERTa, which was fine-tuned on GoEmotions human annotations, is being evaluated against

Gemini-generated labels that follow different standards. Because both models are evaluated on essentially foreign ground truth, the contextual advantage RoBERTa gained was erased. Furthermore, the final dataset consists exclusively of political Reddit comments, which possess a different sentiment distribution than the general GoEmotions dataset. Political text tends to be heavily charged and sarcastic in ways that do not map cleanly onto our three bucket schema, contributing to the significant drop in absolute accuracy across both models.

The topic model



Above is the performance for the combined baseline model on the final dataset. The partial credit score was calculated by awarding the following for an individual protection: 100% accuracy for getting all 3 correct, 66.67% accuracy for getting 2 correct, 33.33% for 1 correct prediction, and 0% for 0 correct predictions. The model can get a maximum of 3 correct predictions because we are evaluating sentiment, political lean, and topic, which each contribute one prediction each.



Above is the performance for the combined exploratory model on the final dataset using the same partial credit scoring method described previously.

Overall, we saw a notable improvement in our combined exploratory model, with a 9% improvement on the combined model using the partial credit score system (45.8% vs 36.5%). We see our combined exploratory model is also extremely better at getting all three predictions correct (10.4% vs 1.8%), which is important in real world applications where a model has to be completely correct in its evaluation (rather than a partial result). It was also notable that getting complete failures (zero correct predictions) was more rare for our exploratory model (16.7% vs 23.1%), showing that it was also more reliable than the baseline at achieving some sort of result. This means the exploratory pipeline is both more likely to get everything right and less likely to get everything wrong, showing improvement at both extremes of performance.

5. Lessons Learned and Insights [at least ½ a page]

Political Classification Model Lessons

- The most important lesson learned from the political model was that pre-trained models trained on a different task distribution can perform worse than simple baselines. Zero-Shot politicalBiasBERT, despite being a large transformer model, barely outperformed random guessing (53%) on our balanced dataset due to the mismatch between its 3-class training objective and our binary task. This was surprising as I expected a politically pre-trained BERT model to have a much larger advantage even without fine-tuning (just 3% over guessing despite all this knowledge), but the neutral class interference was significant enough to undermine almost all political knowledge it had learned.

- Fine-tuning proved to be the single most impactful improvement, jumping from 53% to 73% simply by replacing the classification head and training on our binary dataset. Hyperparameter tuning then added a further 5%, with the most notable gains coming from increasing the max sequence length, suggesting that longer posts with text bodies contain more distinguishing political signals that the shorter sequence length was previously cutting off.
- The generalization gap between our original test set (78%) and the final dataset (54%) highlighted a key limitation of subreddit based labeling; models trained in this manner learn subreddit specific language patterns rather than universal political ideology. A post from r/conservatives uses different vocabulary than a post from a broader r/politics thread, even if both express conservative viewpoints. Future work could address this by training on a more diverse set of subreddits or incorporating cross-subreddit validation during training. Additionally, the dataset's age difference (2022 for our original dataset vs 2026 for our final one) shows that political language is constantly changing, so we need to constantly scrape and retrain models on newer political data to yield accurate results.

Sentiment Classification Model lessons

- Extremely rare emotions, such as grief (6 test samples) and relief (11 test samples), are nearly impossible for the model to learn or for us to evaluate reliably. Initially, using linear class weighting (neg_count/pos_count) penalized these classes and made an unstable training process. Switching to a square root dampening approach helped while still upweighting minority classes to maintain recall, acting as a fix that noticeably improved our validation macro F1.
- The largest performance gains from full RoBERTa fine-tuning were observed on multi-clause sentences, political sarcasm, and comments where the emotional meaning depends heavily on full context. Short or single-word comments showed the least improvement and are inherently difficult for any model to classify.
- Applying a uniform 0.5 sigmoid threshold across all 28 emotion classes significantly underperformed in our multi-label setup. After tuning individual per-class thresholds on the validation set, our macro F1 greatly improved. The optimized thresholds spanned from 0.30 for rare emotions like surprise and relief up to 0.63 for gratitude demonstrating that different emotions require different confidence calibrations. Future work could explore more principled calibration alternatives.
- Across all of our evaluated models, the Negative sentiment class consistently had the lowest F1 score. The negative emotion cluster (encompassing anger, annoyance, disappointment, disapproval, disgust, etc.) shares overlapping vocabulary, making the boundary between "negative" and "neutral" blurry. If this project were extended, negative sub-classification would be the most valuable area for improvement.

Topic Classification Model lessons

- A major issue that kept reoccuring when it came time to evaluate the BERTopic model was the inaccuracy of the models labeling compared to the LLM labels. Initially the BERTopic model would label a majority of the reddit comments as outlier comments which is why we had a high initial accuracy in the beginning. While the accuracy looked good on paper it just meant it would label a comment as an outlier more than 50% of the time which doesn't help classification

- To combat this we used K-means clustering to force every comment into a cluster and remove the need for an outlier topic. The accuracy decreased drastically down to almost 22%, while now it was giving more specific labels, the labels themselves were not as accurate.
- We credited this drastic inaccuracy to the idea of politics, and how our test data set was a scraping of reddit comments on the current day (February- March 2026) r/politics subreddit. Meanwhile the data sets that we were training on were outdated in comparison, where the 1 million reddit comments r/politics subreddit was from May 2019. This mean that a lot of political topics discussed at the time are not being discussed now which makes our model very dated and more inaccurate
- To combat this and improve accuracy we also made the custom topic labels more broad/vague so the LLM was more generous in labeling comments allowing our model to have a higher chance at improving accuracy.
- Next time if we wanted to make sure our Topics were highly accurate and specific we would train on scrapped data from the r/politics subreddit.

Combined Model Lessons

- Combining three independently trained models into a pipeline revealed that the weakest model acts as a bottleneck for the entire system. The baseline pipeline's chance of getting all three predictions correct (1.8%) was heavily dragged down by the topic model performing at only 9%, even though the sentiment and political models were both around 50%. This demonstrates that in a multitask pipeline, overall performance is not simply the average of individual model performance; a single poorly performing component notably reduces the combined model's performance.
- The nearly 6x improvement in getting all 3 predictions correct (1.8% to 10.4%) from baseline to exploratory pipeline was bigger than expected given the modest individual model improvements from the political and sentiment models. This suggests individual model improvements compounded positively (especially when in reference to the worst model); when the topic model improved the most (27%), it unlocked many samples that were previously stuck at 1-2 correct into the 2-3 correct buckets.
- The partial credit scoring system is a more informative evaluation metric for multitask pipelines than averaging individual accuracies; it captures and offers additional insight on partial and complete failures/successes
- Exploratory models that utilize advanced AI and ML techniques like fine-tuning and hyperparameter tuning show notable improvements over traditional baseline models in terms of accuracy while sacrificing some speed

APPENDICES

Additional Results and Backup Information (OPTIONAL)

Gemini labeling prompt for the evaluation dataset:

“”””

EMOTION RULES:

- Choose between 1 and 3 emotions from the list below.
- Only select multiple if the comment strongly expresses mixed feelings.
- If unsure or text is very short/unclear, choose ["neutral"].
- Return emotions as a list of strings.
- Provide emotion_confidence (how confident you are about the emotion labels).

EMOTION LIST:

{', '.join(EMOTIONS)}

TOPIC SELECTION RULES:

- You MUST consider ALL topic indices before choosing.
- Internally score EACH topic for relevance:
  0 = not relevant
  1 = weakly relevant
  2 = relevant
  3 = dominant
- Choose the topic with the highest score.
- If there is a tie, choose the LOWER index.
- If no topic scores 1 or higher, choose Index -1 (Outliers/Unclassifiable).
- Overlapping topics are expected.
- Relevance is based on the comment's MAIN CLAIM, not keyword frequency.
- Ignore incidental or passing mentions.
- Provide topic_confidence (how confident you are about the topic choice).

TOPIC LIST:

{formatted_topics}

CONFIDENCE SCORING GUIDELINES:

- 0.9-1.0: Very confident, clear and unambiguous
- 0.7-0.9: Confident, some minor ambiguity
- 0.5-0.7: Moderate confidence, multiple interpretations possible
- 0.3-0.5: Low confidence, very ambiguous
- 0.0-0.3: Very uncertain, essentially guessing

OUTPUT FORMAT:

Return ONLY valid JSON:

```
{{
  "emotions": ["<string>"],
  "topic_index": <integer>,
  "emotion_confidence": <float>,
  "topic_confidence": <float>
}}
"""
```